

BERECHENBARKEIT UND KOMPLEXITÄT

Übungsblatt 7

Prof. Rossmanith
C. Grüne, T. Janßen, T. Koglin
Lehrstuhl für Informatik 1
RWTH Aachen

WS 24/25
03.12.2024
Abgabe: Di. 10.12., 16:30

Lösung

Tutoraufgabe 7.1

Zeigen Sie, dass folgende Befehle durch ein LOOP-Programm simuliert werden können:

- (a) $x_i := x_j \dot{-} 1$ (modifizierte Vorgängerfunktion mit Ergebnis 0, falls $x_j = 0$)

Lösung:

```
1:  $x_i := 0$ ;  
2:  $y := 0$ ;  
3: LOOP  $x_j$  DO  
4:    $x_i := y$ ;  
5:    $y := y + 1$   
6: ENDLOOP
```

Jede Iteration der Schleife berechnet aus (x_i, y) die neuen Werte $(y, y + 1)$, d.h., es wird die Folge $(0, 0), (0, 1), (1, 2), \dots$ berechnet. Beginnend mit $(0, 0)$ liefern x_j Iterationen also das Paar $(x_j - 1, x_j)$.

- (b) $x_i := x_j \dot{-} x_k$ (modifizierte Subtraktion mit Ergebnis 0, falls $x_j < x_k$)

Lösung:

Nach (a) ist die modifizierte Vorgängerfunktion LOOP-berechenbar.

```
1:  $x_i := x_j$ ;  
2: LOOP  $x_k$  DO  
3:    $x_i := x_i \dot{-} 1$   
4: ENDLOOP
```

- (c) $x_i := \min\{x_j, x_k\}$

Lösung:

Nach (b) ist die Subtraktion und nach Vorlesung das IF- $x_i \dot{-} 0$ -THEN-ELSE-Konstrukt LOOP-berechenbar. Weiter gilt $x_j - x_k \leq 0$ gdw. $\min\{x_j, x_k\} = x_j$.

```
1:  $y := x_j \dot{-} x_k$ ;  
2: IF  $y = 0$  THEN  
3:    $x_i := x_j$   
4: ELSE  
5:    $x_i := x_k$   
6: ENDIF
```

- (d) IF $x_i = c$ THEN P_1 ELSE P_2 ENDIF (Für eine Konstante $c \in \mathbb{N}$)

Lösung:

Für eine Konstante $c \in \mathbb{N}$ nutzen wir das Makro aus der Vorlesung, einer Variable eine beliebige Konstante zuzuweisen. Dann berechnen wir $x_i \dot{-} c$ und $c \dot{-} x_i$, um sicherzustellen, dass $x_i = c$, und benutzen dann das IF- $x_i \dot{-} 0$ -THEN-ELSE-Konstrukt aus der Vorlesung.

Hinweis: Man kann in dieser Konstruktion c auch durch eine weitere Eingabevariable ersetzen, um eine IF-Bedingung zu erhalten, die den Wert der beiden Variablen miteinander vergleicht.

```

1:   $x_j := c;$ 
2:   $y := x_i \dot{-} x_j;$ 
3:   $z := x_j \dot{-} x_i;$ 
4:   $y := y + z;$ 
5:  IF  $y = 0$  THEN  $P_1$  ELSE  $P_2$  ENDIF

```

(e) $x_i := x_j \text{ DIV } x_k$ (Division ohne Rest, gegeben $x_k > 0$)

Lösung:

Wir suchen das kleinste $x_i \leq x_j$ so dass $(x_i + 1) \cdot x_k > x_j$. Dazu probieren wir alle x_i von 0 bis x_j aus. Daher, falls wir ein x_i mit $(x_j + 1) - (x_i \cdot x_k) \leq 0$ finden, ist das Ergebnis $(x_i - 1)$. Wir beginnen daher mit $x_i = 1$ und dem Test von $d := (x_j + 1) - x_k = 0$. Für nachfolgende Werte von x_i , können wir d um x_k vermindern.

```

1:   $x_i := 1;$ 
2:   $d := x_j + 1;$ 
3:   $d := d \div x_k;$ 
4:  LOOP  $x_j$  DO
5:      IF  $d = 0$  THEN ELSE
6:           $x_i := x_i + 1;$ 
7:           $d := d \div x_k;$ 
8:      ENDIF
9:  ENDLOOP;
10:  $x_i := x_i \div 1$ 

```

Hinweis: Wir benutzen in der IF-Anweisung, dass das leere Programm auch ein LOOP-Programm ist. Es genügen x_j Iterationen der LOOP-Schleife, da $(x_j + 1) \cdot x_k > x_j$ gilt. Im Fall $x_k = 1$ ist diese Anzahl auch tatsächlich notwendig.

(f) $x_i := x_j \text{ MOD } x_k$ (Modulo, gegeben $x_k > 0$)

Lösung:

Wir benutzen, dass wir in der vorigen Teilaufgabe gezeigt haben, dass es ein LOOP-Programm gibt, das die Division ohne Rest berechnet. Zudem ist nach Vorlesung die Multiplikation LOOP-berechenbar und nach Turaufgabe 7.1 (b) auch die Subtraktion. Es gilt: $x_j \text{ MOD } x_k = x_j - (x_j \text{ DIV } x_k) \cdot x_k$.

- 1: $y := x_j \text{ DIV } x_k;$
- 2: $y := y \cdot x_k;$
- 3: $x_i := x_j \dot{-} y$

Tutoraufgabe 7.2

Wir betrachten das LOOP-Programm P :

```

1: LOOP  $x_1$  DO
2:   LOOP  $x_2$  DO
3:     LOOP  $x_3$  DO
4:        $x_4 := x_4 + 1$ 
5:     ENDLOOP
6:   ENDLOOP
7: ENDLOOP

```

(a) Bestimmen Sie eine natürliche Zahl m_P , sodass $F_P(n) < A(m_P, n)$ für alle $n \in \mathbb{N}$ gilt.

Lösung:

Um ein m zu bestimmen, verwende die Abschätzungen aus dem Induktionsbeweis aus der Vorlesung.

$$\left. \begin{array}{l} 1: \text{ LOOP } x_1 \text{ DO} \\ 2: \quad \text{ LOOP } x_2 \text{ DO} \\ 3: \quad \quad \text{ LOOP } x_3 \text{ DO} \\ 4: \quad \quad \quad x_4 := x_4 + 1 \quad \} < A(2, n) \\ 5: \quad \quad \text{ ENDL O O P} \\ 6: \quad \text{ ENDL O O P} \\ 7: \text{ ENDL O O P} \end{array} \right\} < A(3, n) \left. \right\} < A(4, n) \left. \right\} < A(5, n)$$

Also gilt $F_P(n) < A(5, n)$.

- (b) Zeigen Sie, dass die Wachstumsfunktion $F_P : \mathbb{N} \rightarrow \mathbb{N}$ des LOOP-Programms P die Beziehung $F_P(n) = \Theta(n^3)$ erfüllt.

Lösung:

P übersetzt einen Eingabevektor (a_1, a_2, a_3, a_4) in den Ausgabevektor $(a_1, a_2, a_3, a_4 + a_1 \cdot a_2 \cdot a_3)$. Folglich gilt $f_P(a_1, a_2, a_3, a_4) = \prod_{i=1}^3 a_i + \sum_{i=1}^4 a_i$, und somit

$$F_P(n) = \max \left\{ \prod_{i=1}^3 a_i + \sum_{i=1}^4 a_i : a_1, \dots, a_4 \in \mathbb{N} \text{ mit } \sum_{i=1}^4 a_i \leq n \right\}.$$

Damit gilt $F_P(n) \leq n^3 + 4n$, also $F_P(n) = \mathcal{O}(n^3)$.

Es bleibt zu zeigen, dass $F_P(n) = \Omega(n^3)$. Sei $n \in \mathbb{N}$. Wähle $a_1 = a_2 = a_3 = \lfloor \frac{n}{3} \rfloor$ und $a_4 = n - (a_1 + a_2 + a_3)$. Dann gilt $f_P(a_1, a_2, a_3, a_4) \geq \lfloor \frac{n}{3} \rfloor^3 = \Omega(n^3)$. Daher auch $F_P(n) = \Omega(n^3)$.